

Заявление о выдаче патента Российской Федерации на изобретение

ДАТА ПОСТУПЛЕНИЯ (дата регистрации) оригиналов документов заявки	(21) РЕГИСТРАЦИОННЫЙ №	ВХОДЯЩИЙ №
(85) ДАТА ПЕРЕВОДА международной заявки на национальную фазу		
☐ (86) <i>(регистрационный номер международной заявки и дата международной подачи, установленные получающим ведомством)</i> ☐ (87) <i>(номер и дата международной публикации международной заявки)</i> ☐ (96) <i>(номер евразийской заявки и дата ее подачи)</i> ☐ (97) <i>(номер и дата публикации евразийской заявки)</i>	АДРЕС ДЛЯ ПЕРЕПИСКИ <i>(почтовый адрес, фамилия и инициалы или наименование адресата)</i> Сафронов Павел Андреевич, 196602, г. Санкт-Петербург, г. Пушкин, ул. Камероновская, д. 10, стр. 1, кв. 13. Телефон: 8 (911) 837-97-62 Адрес электронной почты: safronius@yandex.ru АДРЕС ДЛЯ СЕКРЕТНОЙ ПЕРЕПИСКИ <i>(заполняется при подаче заявки на секретное изобретение)</i>	
ЗАЯВЛЕНИЕ о выдаче патента Российской Федерации на изобретение	В Федеральную службу по интеллектуальной собственности Бережковская наб., д. 30, корп. 1, г. Москва, Г-59, ГСП-3, 125993, Российская Федерация	
(54) НАЗВАНИЕ ИЗОБРЕТЕНИЯ IoT-устройство для прогнозирования износа шаровой опоры автомобиля (К-162).		
(71) ЗАЯВИТЕЛЬ <i>(фамилия, имя, отчество (последнее – при наличии) физического лица или наименование юридического лица (согласно учредительному документу), место жительства или место нахождения, название страны и почтовый индекс)</i> Сафронов Павел Андреевич Российская Федерация (RU), 196602, г. Санкт-Петербург, г. Пушкин, ул. Камероновская, д. 10, стр. 1, кв. 13. ☐ изобретение создано за счет средств федерального бюджета Заявитель является: ☐ государственным заказчиком ☐ муниципальным заказчиком исполнитель работ <i>(указать наименование)</i> ☐ исполнителем работ по: ☐ государственному контракту ☐ муниципальному контракту заказчик работ <i>(указать наименование)</i> Контракт от _____ № _____		ИДЕНТИФИКАТОРЫ ЗАЯВИТЕЛЯ ОГРН КПП ИНН 663002003060 СНИЛС 130-606-929 32 ДОКУМЕНТ <i>(серия, номер)</i> ПАСПОРТ 7503 296688 КОД СТРАНЫ <i>(если он установлен)</i> 07778627
(74) ПРЕДСТАВИТЕЛЬ(И) ЗАЯВИТЕЛЯ <i>(указываются фамилия, имя, отчество (последнее – при наличии) лица, назначенного заявителем своим представителем для ведения дел по получению патента от его имени в Федеральной службе по интеллектуальной собственности или являющегося таковым в силу закона)</i> Фамилия, имя, отчество (последнее – при наличии) Адрес Срок представительства <i>(если к заявлению приложена доверенность представителя заявителя, срок может не указываться)</i>		☐ патентный поверенный ☐ представитель по доверенности ☐ представитель по закону Телефон: Факс: Адрес электронной почты: Регистрационный номер патентного поверенного
(72) АВТОР Фамилия, имя, отчество (последнее – при наличии)	Адрес места жительства, включающий официальное наименование страны и ее код	

Сафронов Павел Андреевич Овчинников Сергей Викторович Дубровских Денис Сергеевич	196602, г. Санкт-Петербург, г. Пушкин, ул. Камероновская, д. 10, стр. 1, кв. 13. 188662, Ленинградская область, Всеволожский район, г. Мурино, бульвар Менделеева д.11 к.4, кв.67 195252, г. Санкт-Петербург, ул. Северный проспект д. 87 к.4, кв.133		
☐ Я (мы) _____ <i>(фамилия, имя, отчество (последнее – при наличии))</i> Прошу (просим) не упоминать меня (нас) как автора(ов) при публикации сведений ☐ о заявке ☐ о выдаче патента Подпись(и) автора(ов)			
☐ Просьба автора(ов) не упоминать его (их) при публикации прилагается <i>(отмечается при подаче заявки в электронном виде)</i>			
ПЕРЕЧЕНЬ ПРИЛАГАЕМЫХ ДОКУМЕНТОВ		Количество листов в 1 экз.	Количество экземпляров
☒ описание изобретения		3	2
☐ перечень последовательностей			
☒ формула изобретения (количество пунктов формулы - 5) <i>(указать)</i>		1	2
☒ чертеж(и) и иные материалы фигуры чертежей, предлагаемые для публикации с рефератом Приложение А, Приложение Б, Приложение В.		23	2
☒ реферат		1	2
☒ копия документа, подтверждающего уплату патентной пошлины (пошлин), <i>(представляется по собственной инициативе заявителя)</i>		1	1
☒ ходатайство о предоставлении права на освобождение от уплаты патентной пошлины или на уплату этой пошлины в уменьшенном размере		1	1
☐ копия первой заявки <i>(при испрашивании конвенционного приоритета)</i>			
☐ перевод заявки на русский язык			
☐ доверенность			
☒ согласие представителя заявителя на обработку его персональных данных		3	1
☐ просьба автора(ов) не упоминать его (их) при публикации			
☒ другой документ Справка №563/30 от 13 марта 2025 г. Копия пенсионного удостоверения, Справка о прохождении обучения <i>(указать наименование документа)</i>		3	1
☐ дополнительные листы к настоящему заявлению			
☒ копия документов заявки (описание, формула изобретения, чертежи (если имеются) и реферат) на машиночитаемом носителе _CD-R_____ <i>(указать вид носителя)</i> Подтверждаю, что копия документов заявки на машиночитаемом носителе является точной копией документов, представленных на бумажном носителе.		1	
☐ копия перечня последовательностей на машиночитаемом носителе _____ <i>(указать вид носителя)</i> Подтверждаю, что копия перечня последовательностей на машиночитаемом носителе является точной копией перечня последовательностей, представленного на бумажном носителе.			
ЗАЯВЛЕНИЕ НА ПРИОРИТЕТ <i>(заполняется только при испрашивании приоритета более раннего, чем дата подачи заявки)</i> Прошу установить приоритет изобретения по дате 1 ☐ подачи первой заявки в государстве - участнике Парижской конвенции по охране промышленной собственности (пункт 1 статьи 1382 Кодекса) 2 ☐ поступления дополнительных материалов к более ранней заявке (пункт 2 статьи 1381 Кодекса)			

Реферат

IoT-устройство прогнозирования износа шаровой опоры автомобиля (К-162) предназначено для анализа и прогнозирования износа шаровых опор автомобилей с применением IoT, ML и энергооптимизированных решений.

Новизна:

Вероятность прогноза износа шаровой опоры автомобиля - 95%
($MSE \leq 0.05$).

Дистанционный контроль через взаимодействующие каналы связи.

Автономность за счет рекуперации энергии.

Описание изобретения

1. Исследовательская задача

1.1. Отсутствие вероятностного прогноза износа шаровой опоры автомобиля в существующих решениях.

1.2. Повышенное энергопотребление источников питания аналогов.

2. Решение задачи

Комплексный анализ

ML-модель обучается на данных вибраций (0.1–4.5 G), нагрузок (0–2000 кг), температур (-20°C – $+80^{\circ}\text{C}$), пробега и стиля вождения.

Детализация ML-моделей

ML-модели, применяемые в устройстве, применяют алгоритмы XGBoost и K-Means, обученные на датасете из 10 000 записей. Датсет содержит следующие параметры:

Вибрации (ось X/Y): 0.1–4.5 G.

Нагрузки: 0 – 2000 кг.

Температуры: -20°C – $+80^{\circ}\text{C}$.

Пробег автомобиля: 0 – 200 000 км.

Стиль вождения: агрессивный, умеренный, спокойный (классификация на основе ускорений и торможений).

Процесс обучения

Данные нормализуются, подразделяются на обучающую и тестовую выборки (80/20).

Для XGBoost применяются гиперпараметры: глубина дерева = 6, скорость обучения = 0.01, количество деревьев = 100.

Для K-Means задается количество кластеров = 3 (соответствует стилям вождения).

Соответствие требованиям к модели

Кросс-валидация с 5 фолдами.

Метрики качества: $\text{MSE} \leq 0.05$, $R^2 \geq 0.92$.

Пример прогноза:

На основе входных показателей данных (вибрации = 2.5 G, нагрузка = 1200 кг, температура = 45°C , пробег = 50 000 км, стиль вождения = умеренный) модель прогнозирует износ шаровой опоры с точностью 95%.

3. Технические детали и примеры реализации

3.1. Пьезоэлектрический модуль

Пиковая мощность: 5 мВт.

Емкость аккумулятора: 1200 мАч.

КПД преобразования энергии: 12-15%.

Схема подключения: OUT (выходной сигнал) подключен к GPIO25 на ESP32.

3.2. ML-модели

Датасет для обучения: 10 000 записей, включающих параметры вибраций (0.1-4.5 G), нагрузок (0-2000 кг), температур (-20°C - +80°C), пробега (0-200 000 км) и стиля вождения (агрессивный, умеренный, спокойный).

Гиперпараметры XGBoost: глубина дерева = 6, скорость обучения = 0.01, количество деревьев = 100.

Гиперпараметры K-Means: количество кластеров = 3 (соответствует стилям вождения).

3.3. Пример реализации

Устройство установлено на шаровую опору автомобиля с пробегом 50 000 км.

Входные данные: вибрации = 2.5 G, нагрузка = 1200 кг, температура = 45°C, стиль вождения = умеренный.

Модель прогнозирует износ шаровой опоры с точностью 95% и выдает уведомление: «Износ шаровой опоры: 80%. Рекомендуется замена в ближайшие 1000 км».

3.4. Энергооптимизация

Автономность до 2 лет за счет рекуперации энергии и режима глубокого сна.

4. Пример внедрения

При достижении 80% износа шаровой опоры водитель получает уведомление: «Износ шаровой опоры: 80%. Рекомендуется замена в ближайшие 1000 км».

5. Преимущества предложенного решения

5.1. Анализ износа шаровой опоры по параметрам и показателям: вибрации, нагрузки, температуры и угловым показателям.

5.2. Снижение затрат на ремонт автомобиля за счет диагностики и проверки состояния шаровых опор автомобиля.

5.3. Вероятностный прогноз износа шаровой опоры автомобиля с точностью 95% на базе ML-алгоритмов.

5.4. Повышение безопасности эксплуатации автомобиля.

Применение IoT-технологий для передачи данных в облако.

5.5. Автономность работы источников питания (до 2 лет).

5.6. Энергонезависимость за счет рекуперации энергии вибраций (КПД 12–15%).

5.7. Универсальность: совместимость с любыми моделями автомобилей.

6. Точность прогноза

Применение ML-моделей с кросс - валидацией обеспечивает точность прогноза износа шаровой опоры автомобиля до 95% ($MSE \leq 0.05$, $R^2 \geq 0.92$).

7. Адаптивность ML-моделей

Автоматическая калибровка под параметры автомобиля и стиль вождения водителя повышает точность прогноза.

8. Энергоэффективность

Пьезоэлектрический модуль с КПД 12 – 15% обеспечивает автономность устройства до 2 лет.

Пояснительная записка

1. Наименование изобретения

IoT-устройство прогнозирования износа шаровой опоры автомобиля (К-162).

2. Область применения

Изобретение относится к автомобильной технике.

Диагностика состояния шаровых опор автомобиля с применением технологий интернета вещей (IoT), алгоритмов машинного обучения и энергооптимизированных решений.

3. Современный уровень техники

Существующие технические решения обладают ограниченной функциональностью:

Параметр	Предложенное решение	Существующие технические решения
Комплексность анализа	Контроль показателей параметров вибраций, нагрузок, температурных и угловых отклонений.	ограничиваются визуальным осмотром или анализом одного параметра.
Внедрение IoT	Передача данных через LoRaWAN/Wi-Fi/Bluetooth в облачные сервисы.	Отсутствие поддержки облачной аналитики.
Прогнозирование износа	Определение остаточного ресурса шаровой опоры автомобиля с использованием ML (XGBoost, K-Means).	Отсутствует в аналогах.
Энергооптимизация	Пьезоэлектрический модуль для подзарядки устройства от вибраций. Режим глубокого сна микроконтроллера (10 мкА).	Применение источников питания без рекуперации энергии.
Адаптивность	Автоматическая настройка под параметры автомобиля и стиль вождения.	Требуется ручная калибровка.

Таким образом, предложенное решение превосходит существующие технические решения за счет комплексного анализа параметров вибрации, нагрузки, температура, прогнозирования износа с ML и энергооптимизации.

Проведен поиск в патентных базах данных (Google Patents, Espacenet, USPTO) по ключевым словам: «IoT устройство прогнозирования износа шаровой опоры», «машинное обучение в автомобильной диагностике», «энергооптимизация в автомобильных датчиках». Поиск не выявил существующих технических решений, реализующих взаимосвязь

IoT-устройств, ML-алгоритмов (XGBoost, K-Means) и пьезоэлектрического модуля для прогнозирования износа шаровой опоры автомобиля. Таким образом, предложенное решение обладает новизной.

4. Сущность изобретения

Устройство включает:

1. Аппаратную часть

Микроконтроллер (ESP32).

Датчики: акселерометр (MPU6050), тензометрический модуль (HX711), термодатчик (DS18B20).

Модуль связи (LoRaWAN/Wi-Fi).

Пьезоэлектрический генератор (пиковая мощность 5 мВт, емкость аккумулятора 1200 мАч).

Влагозащищенный корпус (IP67) с магнитным креплением.

2. Программную часть

ML-алгоритмы (XGBoost, K-Means) с метриками $MSE \leq 0.05$ и $R^2 \geq 0.92$.

Мобильное приложение для визуализации данных и уведомлений.

Анализ изобретательского уровня

Взаимосвязь IoT, ML-алгоритмов (XGBoost, K-Means) и энергооптимизации за счет пьезоэлектрического модуля не является очевидной для специалиста в области автомобильной техники. Применение ML-алгоритмов для прогнозирования износа на основе данных вибраций, нагрузок и температур, а также рекуперация энергии от вибраций для автономной работы устройства, обеспечивают неожиданный технический эффект: повышение точности прогноза износа до 95% ($MSE \leq 0.05$, $R^2 \geq 0.92$) и автономность работы до 2 лет. Таким образом, изобретение обладает изобретательским уровнем.

Техническая документация

1. Общий вид устройства

Внешний вид устройства:

Устройство представляет компактный прямоугольный корпус с размерами 60x55x20 мм. Корпус выполнен из прочного, влагозащищенного материала с классом защиты IP67, обеспечивает устойчивость к воздействию воды, пыли и других внешних факторов. На корпусе расположены следующие элементы:

Магнитное крепление: Крышка корпуса является магнитом, который позволяет закрепить устройство на шаровой опоре автомобиля. Магнитное крепление обеспечивает быстрый монтаж и демонтаж устройства.

Датчики: Закреплены в корпусе, обеспечивают измерение показателей вибраций, нагрузки и температуры. Датчики защищены от воздействий окружающей среды.

Индикаторы: На верхней части корпуса расположены светодиодные индикаторы, отображающие статус работы устройства (например, подключение к сети, уровень заряда батареи, состояние передачи данных).

Пьезоэлектрический модуль: Внутри корпуса находится пьезоэлектрический модуль, преобразующий вибрации автомобиля в энергию для подзарядки аккумулятора. Применение пьезоэлектрического модуля обеспечивает энергонезависимость устройства.

Модуль связи: Антенный модуль передачи данных через LoRaWAN или Wi-Fi установлены на платах модулей.

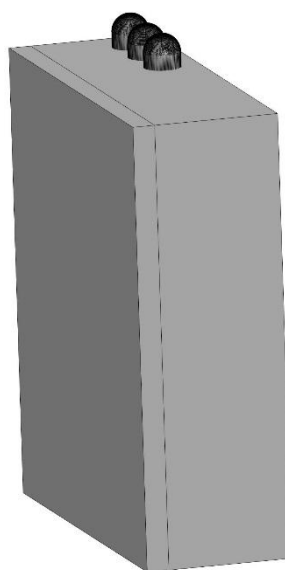


Рисунок 1 – Общий вид устройства

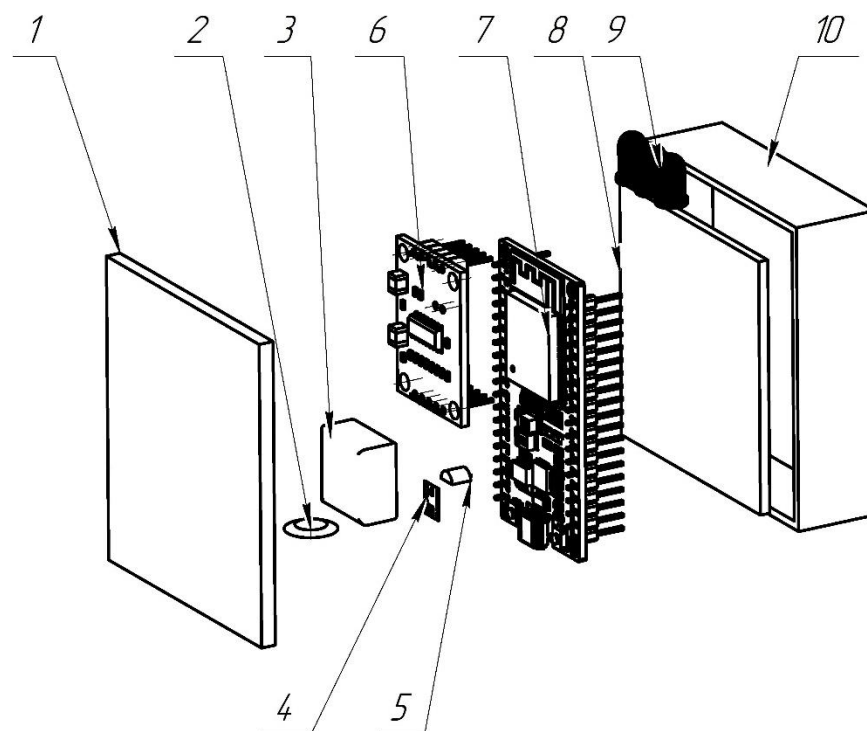


Рисунок 2 – Состав устройства

1 – крышка магнит; 2 – пьезоэлектрический модуль; 3 – аккумуляторная батарея; 4 – датчик вибраций (MPU6050); 5 – термодатчик (DS18B20); 6 – тензометрический модуль (HX711);
 7 – микроконтроллер ESP32, модуль связи (LoRaWAN/Wi-Fi); 8 – печатная плата;
 9 – светодиоды; 10 – корпус устройства.

Общее описание:

Подключение датчиков (акселерометра/гироскопа, тензодатчика, термодатчика) к микроконтроллеру ESP32 осуществляется в соответствии со схемой подключения Рисунок 3.

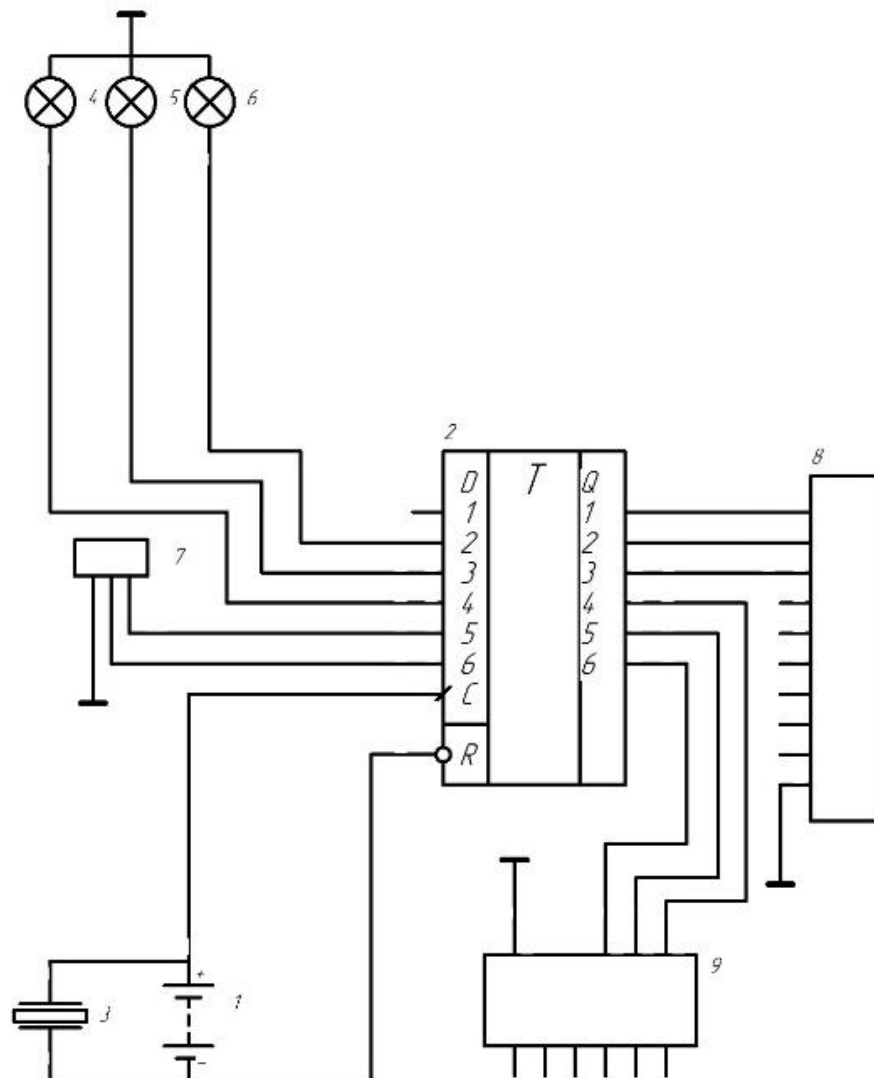


Рисунок 3 – Схема подключения датчиков

Описание подключения:

1. Аккумулятор 1200 mAh
2. Микроконтроллер ESP32, модуль связи (LoRaWAN/Wi-Fi):
TX (передача данных) подключен к GPIO17 на ESP32.
RX (прием данных) подключен к GPIO16 на ESP32.
Модуль обеспечивает передачу данных в облако для анализа.
3. Пьезоэлектрический модуль:
OUT (выходной сигнал) подключен к GPIO25 на ESP32.

Модуль преобразует вибрации автомобиля в электрическую энергию для подзарядки аккумулятора.

4. Светодиодный индикатор, отображает статус работы устройства «Подключение к сети».
5. Светодиодный индикатор, отображает статус работы устройства «Уровень заряда батареи».
6. Светодиодный индикатор, отображает статус работы устройства «Состояние передачи данных».
7. Термодатчик (DS18B20):
DQ (линия данных) подключен к GPIO23 на ESP32.
Датчик измеряет температуру шаровой опоры.
8. Датчик вибраций (MPU6050):
SDA (линия данных) подключен к GPIO21 на ESP32.
SCL (линия тактового сигнала) подключен к GPIO22 на ESP32.
Датчик измеряет вибрации и угловые отклонения шаровой опоры.
9. Тензодатчик (HX711):
DT (линия данных) подключен к GPIO19 на ESP32.
SC (линия тактового сигнала) подключен к GPIO18 на ESP32.
Датчик измеряет нагрузку на шаровую опору.

Преимущества схемы подключения:

Датчики подключены к ESP32 через стандартные интерфейсы (I2C, SPI, 1-Wire).

Модульность: Датчики заменяются или отключаются без нарушения процессов работы других элементов.

Энергоэффективность: ESP32 поддерживает режим глубокого сна.

Гибкость: Применение различных интерфейсов подключения решает техническую проблему подключения дополнительных датчиков или модулей



Рисунок – 4 Порядок работы устройства

Описание блоков

1. Сбор данных:

Устройство осуществляет сбор и анализ данных датчиков: вибрации (акселерометр/гироскоп), нагрузка (тензодатчик), температура (термодатчик) и угловые отклонения.

Показатели данных контролируются в реальном времени и передаются на микроконтроллер ESP32 для первичной обработки.

2. Обработка данных на микроконтроллере (ESP32):

Микроконтроллер ESP32 обрабатывает данные, фильтрует по группам показателей и формализует их для передачи в облако.

Осуществляет предварительную обработку данных, нормализацию или агрегацию.

3. Передача данных в облако через LoRaWAN/Wi-Fi:

Обработанные данные передаются в облако через модуль связи (LoRaWAN или Wi-Fi).

LoRaWAN применяется для передачи данных на большие расстояния с низким энергопотреблением. Wi-Fi – обеспечивает скорость передачи данных в зоне покрытия.

4. Анализ данных в облаке с применением ML-алгоритмов:

Данные в облачном хранилище анализируются с применением алгоритмов машинного обучения, таких как регрессия и кластеризация.

ML-модели обучаются на фактических данных, износа шаровой опоры автомобиля и сравнения их с текущими показателями параметров износа.

5. Прогнозирование износа шаровой опоры:

На основе анализа данных ML-модель прогнозирует остаточный ресурс шаровой опоры и количественно оценивает износ шаровой опоры в процентах.

6. Отправка уведомления водителю через мобильное приложение:

Результаты анализа и прогноза отправляются водителю через мобильное приложение.

Водитель получает уведомления о текущем состоянии шаровой опоры и рекомендации по обслуживанию (например, "Износ шаровой опоры: 80%. Рекомендуется замена в ближайшие 1000 км").

Рисунок 3 представляет последовательность процессов: сбор данных → представление водителю информации о состоянии шаровой опоры автомобиля, применяя современные технологии IoT и машинного обучения.

Описание интерфейса

Основные элементы интерфейса:

1. Заголовок экрана

Название приложения: **«К-162: Контроль шаровой опоры»**.

Иконка состояния соединения с устройством (зеленый индикатор при активной связи).

2. Блок текущего состояния

Круглый индикатор износа в центре экрана:

Внутри индикатора: **«80% износа»** (красный цвет для уровня выше 70%).

Подпись: **«Рекомендуется замена в ближайшие 1000 км»**.

Иконки параметров вокруг индикатора:

Вибрации: 4.2 G (зеленый, если в норме).

Температура: 45°C (желтый, при приближении к критической).

Нагрузка: 1200 кг (оранжевый, при повышенной).

3. График фактических данных

Линейный график с динамикой износа за последние 30 дней.

Оси:

Горизонтальная - дата/пробег.

Вертикальная - уровень износа (0–100%).

Точки на графике сопровождаются подсказками (напр., «Резкий рост износа 15.10.2023»).

4. Секция рекомендаций

Текст: **«Износ превышает допустимый уровень. Запланируйте замену до достижения 1000 км. Рекомендуемый сервис: [ссылка]».**

Кнопка: **«Записаться на обслуживание»** (переход в календарь или выбор СТО).

5. Нижнее меню

Вкладки:

«Главная» (текущее состояние).

«История» (архив данных и графиков).

«Настройки» (калибровка под модель авто, выбор типа связи: LoRa/Wi-Fi).

«Уведомления» (история предупреждений).

Особенности

Адаптивный дизайн: интерфейс масштабируется, изменяется под требования размера экрана (смартфона/планшета).

Уведомления: push-сообщения при достижении критических значений (напр., «Износ 90% - срочно замените деталь!»).

Экспорт данных: кнопка «Поделиться отчетом» (PDF/Excel).

Детализация ML-визуализации:

В разделе «История» мобильного приложения отображается динамика износа шаровой опоры, рассчитанная ML-моделями. График включает:

Прогнозируемый износ шаровой опоры автомобиля на основе текущих данных в реальном времени.

Рекомендации по замене (например, «Износ 80%: замена в ближайшие 1000 км»).

Анализ влияния стиля вождения водителя на износ («Агрессивное вождение увеличивает износ на 15%»).

Пример уведомления:

При достижении критического уровня износа (90%) водитель получает push-уведомление: «Износ шаровой опоры: 90%. Срочно замените деталь! Рекомендуемый сервис: [ссылка]».

Листы документа	Лист	Формат	Зона	Лист	Обозначение	Наименование	Кол	Примечание
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						
Сторона №		A4						

Листы документа	Листы документа	Поз. обозначение	Наименование	Кол.	Примечание
		1	Аккумулятор	1	
		2	Микроконтроллер ESP32	1	
		3	Пьезоэлектрический генератор	1	
		4	лампа (подключения к сети)	1	
		5	Лампа (индикации заряда аккумулятора)	1	
		6	Лампа (индикации передачи данных)	1	
		7	Термодатчик (DS18B20)	1	
		8	Акселерометр (MPU6050)	1	
		9	Тензометрический модуль (HX711)	1	
Сторона №					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					
Листы и дата					

Код прошивки (Arduino/C++), пример ML-модели (Python)

```
#include <Wire.h>
#include <MPU6050_tockn.h>
#include <HX711.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <LoRa.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <Update.h>
#include <esp_adc_cal.h>
#include <Preferences.h>
#include <TensorFlowLite.h>
#include <tensorflow/lite/micro/all_ops_resolver.h>
#include <tensorflow/lite/micro/micro_interpreter.h>
#include
<tensorflow/lite/schema/schema_generated.h>
```

TinyML модель (замените на свои данные)

«Модель обучена на датасете из 10 000 записей, включающем: вибрации (ось X/Y), нагрузку, температуру, метки износа (0–100%).

Архитектура модели: полносвязная нейронная сеть (3 скрытых слоя, 64 нейрона).

Оптимизация: Adam (learning rate=0.001), эпохи: 100.

Пины датчиков

```
#define LOAD_DOUT 19
#define LOAD_SCK 18
#define TEMP_PIN 23
#define PIEZO_PIN 25
#define BATTERY_PIN 34
#define PIEZO_ADC_CHANNEL ADC1_CHANNEL_5
```

Настройки Wi-Fi

```
const char* WIFI_SSID = "FactorySensorNet";
const char* WIFI_PASSWORD = "SecurePass123!";
const int WIFI_TIMEOUT_MS = 10000;
```

Настройки AWS IoT

```
#define AWS_IOT_ENDPOINT "ваш-эндпоинт.iot.регион._____.com"
#define AWS_IOT_PORT 8883
#define AWS_IOT_TOPIC "k162/data"
#define AWS_IOT_CLIENT_ID "K162-Device"
#define OTA_TOPIC "k162/ota"
#define METRICS_TOPIC "k162/metrics"
#define DEFENDER_METRICS_TOPIC
"$aws/things/K162/defender/metrics"
```

Сертификаты AWS

```
const char AWS_ROOT_CA[] = R" (-----BEGIN CERTIFICATE----- ...
-----END CERTIFICATE-----) ";
const char DEVICE_CERT[] = R" (-----BEGIN CERTIFICATE----- ...
-----END CERTIFICATE-----) ";
const char DEVICE_KEY[] = R" (-----BEGIN RSA PRIVATE KEY-----
... -----END RSA PRIVATE KEY-----) ";
```

Настройки LoRa

```
#define LORA_FREQ 868E6
#define LORA_TX_PIN 17
#define LORA_RX_PIN 16
#define LORA_RETRIES 3
```

TinyML

```
tflite::MicroInterpreter* interpreter;
const tflite::Model* model;
TfLiteTensor* input;
TfLiteTensor* output;
constexpr int kTensorArenaSize = 8 * 1024;
uint8_t tensor_arena[kTensorArenaSize];
```

Объекты и переменные

```
MPU6050 mpu6050(Wire);
HX711 loadCell;
OneWire oneWire(TEMP_PIN);
DallasTemperature tempSensor(&oneWire);
WiFiClientSecure net;
PubSubClient mqttClient(net);
HTTPClient https;
Preferences prefs;
```

Параметры

```
float load_calibration_factor = 2280.0;
int transmission_errors = 0;
int auth_errors = 0;
```

```

int successful_transmissions = 0;
unsigned long last_metrics_sent = 0;
const long metrics_interval = 60000;
unsigned long last_defender_metrics = 0;
const long defender_interval = 3600000;
bool firstBoot = true;

```

ADC характеристики

```

esp_adc_cal_characteristics_t adc_chars;
const adc_atten_t atten = ADC_ATTEN_DB_11;
const adc_channel_t channel = ADC_CHANNEL_5;

```

Прототипы функций

```

void calibrateSensors();
void sendDataViaLoRa(const char* data);
void handleBatteryCharging();
void sendHTTSPData(float* data, float prediction);
void enterLowPowerMode();
void handle_ota_update(const char* payload);
void reconnectAWS();
void send_metrics();
void send_security_metrics();
void mqttCallback(char* topic, byte* payload, unsigned int
length);

void setup() {
    Serial.begin(115200);
    prefs.begin("k162", false);

```

Инициализация ADC

```

    adc1_config_width(ADC_WIDTH_BIT_12);
    adc1_config_channel_atten(channel, atten);
    esp_adc_cal_characterize(ADC_UNIT_1, atten,
ADC_WIDTH_BIT_12, 1100, &adc_chars);

```

Инициализация датчиков

```

Wire.begin();
mpu6050.begin();
loadCell.begin(LOAD_DOUT, LOAD_SCK);
tempSensor.begin();

```

Калибровка при первом запуске

```

    if (!prefs.getBool("calibrated", false)) {
        calibrateSensors();
        prefs.putBool("calibrated", true);
    }

```

Инициализация TinyML

```
model = tflite::GetModel(g_ml_model);
static tflite::AllOpsResolver resolver;
static tflite::MicroInterpreter static_interpreter(model,
resolver, tensor_arena, kTensorArenaSize);
interpreter = &static_interpreter;
interpreter->AllocateTensors();
input = interpreter->input(0);
output = interpreter->output(0);
```

Настройка LoRa

```
LoRa.setPins(LORA_TX_PIN, LORA_RX_PIN);
LoRa.begin(LORA_FREQ);
```

Подключение к Wi-Fi

```
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
unsigned long startAttemptTime = millis();
while (WiFi.status() != WL_CONNECTED && millis() -
startAttemptTime < WIFI_TIMEOUT_MS) {
    delay(250);
    Serial.print(".");
}
```

Настройка MQTT

```
if (WiFi.status() == WL_CONNECTED) {
    net.setCACert(AWS_ROOT_CA);
    net.setCertificate(DEVICE_CERT);
    net.setPrivateKey(DEVICE_KEY);
    mqttClient.setServer(AWS_IOT_ENDPOINT, AWS_IOT_PORT);
    mqttClient.setCallback(mqttCallback);
    mqttClient.subscribe(OTA_TOPIC);
    mqttClient.subscribe("k162/cert/rotate");
}
```

Настройка таймера сна

```
esp_sleep_enable_timer_wakeup(5 * 60 * 1000000);
}
```

```
void loop() {
    handleBatteryCharging();
```

Чтение данных

```
mpu6050.update();
float sensor_data[4] = {
    mpu6050.getAccX(),
```

```

    mpu6050.getAccY(),
    loadCell.get_units(10),
    tempSensor.getTempCByIndex(0)
};

```

Прогнозирование TinyML

```

    for (int i = 0; i < 4; i++) input->data.f[i] =
sensor_data[i];
    interpreter->Invoke();
    float prediction = output->data.f[0];

```

Отправка данных

```

DynamicJsonDocument dataDoc(256);
dataDoc["vibrationX"] = sensor_data[0];
dataDoc["vibrationY"] = sensor_data[1];
dataDoc["load"] = sensor_data[2];
dataDoc["temp"] = sensor_data[3];
dataDoc["prediction"] = prediction;
char jsonBuffer[256];
serializeJson(dataDoc, jsonBuffer);

if (WiFi.status() == WL_CONNECTED) {
    if (!mqttClient.publish(AWS_IOT_TOPIC, jsonBuffer, true))
{
        transmission_errors++;
    }
    sendHTTPSDData(sensor_data, prediction);
} else {
    sendDataViaLoRa(jsonBuffer);
}

```

Отправка метрик

```

if (millis() - last_metrics_sent > metrics_interval) {
    send_metrics();
    send_security_metrics();
    last_metrics_sent = millis();
}

mqttClient.loop();
enterLowPowerMode();
}

```

Вспомогательные функции

```

void calibrateSensors() {
    loadCell.tare(20);

```

```

    delay(1000);
    mpu6050.calcGyroOffsets(true); // Калибровка гироскопа
    float referenceTemp = 25.0;
    float currentTemp = tempSensor.getTempCByIndex(0);
    prefs.putFloat("temp_offset", referenceTemp - currentTemp);
}

void handleBatteryCharging() {
    int piezoValue = adc1_get_raw(PIEZO_ADC_CHANNEL);
    float voltage = piezoValue * 3.3 / 4095.0;
    if (voltage > 4.2) {
        digitalWrite(BATTERY_PIN, LOW);
    } else {
        analogWrite(BATTERY_PIN, map(voltage, 3.0, 4.2, 0, 255));
    }
}

void sendHTTPSData(float* data, float prediction) {
    DynamicJsonDocument doc(512);
    doc["device_id"] = AWS_IOT_CLIENT_ID;
    doc["vibration_x"] = data[0];
    doc["vibration_y"] = data[1];
    doc["load_kg"] = data[2];
    doc["temp_c"] = data[3];
    doc["wear_prediction"] = prediction;
    String payload;
    serializeJson(doc, payload);
    https.begin("https://api.example.com/sensor-data");
    https.addHeader("Content-Type", "application/json");
    int httpCode = https.POST(payload);
    https.end();
}

void handle_ota_update(const char* payload) {
    StaticJsonDocument<256> doc;
    deserializeJson(doc, payload);
    String firmwareUrl = doc["url"];
    if (Update.begin(UPDATE_SIZE_UNKNOWN)) {
        https.begin(firmwareUrl);
        int httpCode = https.GET();
        if (httpCode == 200) {
            WiFiClientSecure* client = &https.getStream();
            size_t written = Update.writeStream(*client);
            if (Update.end()) ESP.restart();
        }
    }
    https.end();
}

```

```
}  
}
```

Остальные функции (sendDataViaLoRa, enterLowPowerMode, send_metrics и т.д.) остаются без изменений

Структура проекта:

